

# PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

First Author, Second Author, Third Author  
ITMO University, St. Petersburg, Russia  
{author1, author2, author3}@itmo.ru

**Abstract**—Features for multi-table prediction tasks are built by aggregating neighbouring tables at a per-row seed time. Correctness requires every aggregate to respect that seed time along every join path and for every column, including columns written to a row *after* the row appears. We show that this is violated routinely, using three sources we did not construct: a widely used feature-engineering library called `without` a cutoff-time table, as in its introductory example (+14.8 to +16.3 AUC points), the reference code written by the authors of this paper (+0.2 to +5.3 points depending on the task), and code generated by two language models under a neutral prompt (53% and 35% of programs that ran). We demonstrate PITFALL, which runs a feature program twice — on the full database and on a database physically truncated at the seed time — and reports any divergence. The check parses nothing, is language-agnostic, has no false positives by construction, names the offending columns in 0.2–7.6s, and, by truncating one availability channel at a time, localises the leak to the table and column it enters through. The univariate probe shipped by commercial AutoML platforms misses all three sources at its default warning threshold and raises a false alarm on a correct pipeline at the stricter one.

**Index Terms**—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code

## I. INTRODUCTION

In multi-table (relational) machine learning a prediction task is given as a table of triples (*entity*, *seed time*, *label*) over a database with foreign keys and timestamps [9]. Predictive features are obtained by walking join paths out of the entity and aggregating neighbouring rows; such features are worth on the order of ten AUC points over the entity table alone [11]. Every aggregate carries an obligation: it may only read facts that existed at the seed time of that particular row.

Leakage is a documented driver of irreproducibility across applied ML [1], [2], and this obligation is easy to state and easy to violate. Truncation is *per row*, so harnesses that cut the database once at the start of the test period say nothing about individual seed times; it must hold along *every* join path, where the governing timestamp often belongs to a neighbour; and — the case systematically missed — a row can carry columns written *later than the row itself*, such as a review attached to an order. Filtering by the order timestamp admits the row and its future columns together.

Our study set out to test whether language-model agents produce leaking feature code; the clearest evidence

turned out to concern standard practice. Violations appear in three independent sources, none constructed by us: `featuretools` [13] called without a cutoff-time table, the form of its introductory example [14]; the reference implementation written by the authors of this paper, found by our own checker; and single-shot generations from two language models under a prompt that never mentions leakage.

Existing safeguards do not cover this case. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker Data Wrangler) ship the same univariate probe — flag a feature whose single-feature AUC exceeds a threshold [7], [8]. Rule-based checkers such as `leakr` [6] enumerate a fixed list of patterns. Static analyzers for notebooks [3] and pipeline instrumentation such as `mlinspect` [4] target preprocessing order and train/test contamination, not temporal leakage. Asking a language model directly performs poorly: low-shot prompting reaches 0.19 precision at 0.52 recall, against 0.86/0.70 for a trained classifier [5].

**Contributions.** (i) *Differential execution* as a correctness check for feature programs: sound, one-sided, independent of language and of any understanding of the code, with leak localisation by per-channel truncation. (ii) Evidence from three independent sources that point-in-time violation is routine. (iii) An execution-verified measurement of the rate at which two language models produce temporally incorrect feature code for a multi-table task, decomposed by task construction and prompt guard strength. (iv) A working system and a four-scene demonstration in which the checker fires where the industrial probe is silent, stays silent where the code is correct, and localises and patches the leak.

## II. POINT-IN-TIME CORRECTNESS

### A. Definition

Let  $D$  be a database,  $t$  a seed time and  $\varphi$  a feature program. Each row  $r$  has a row timestamp  $\text{time}(r)$ ; a column  $c$  of  $r$  may in addition have its own availability timestamp  $\text{avail}_c(r) \geq \text{time}(r)$ . Write  $D|_t$  for the database in which rows with  $\text{time}(r) > t$  are removed and values with  $\text{avail}_c(r) > t$  are replaced by NULL. Then  $\varphi$  is *point-in-time correct* at  $t$  iff

$$\varphi(D, t) = \varphi(D|_t, t). \quad (1)$$

For a cutoff shift  $\delta \geq 0$  we write  $I(\delta)$  for the AUC of a *fixed* learner  $A$  trained and evaluated on features computed with cutoff  $t + \delta$ , minus its AUC at  $\delta = 0$ . Fixing  $A$  matters: in one agent trajectory [12] we examined, swapping the boosting implementation on a bit-identical feature set moved AUC by 11.5 points, the order of the effect measured here.

### B. Availability relation

Equation (1) generalises by replacing the time predicate with an *availability relation*  $R(r, e, t)$ , true when row  $r$  may be read when predicting for entity  $e$  at time  $t$ ; co-emission and group leakage are other masks over the same machinery. Availability is defined *per column*: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days (max 147) after the order row it is attached to. Some columns admit *no* availability timestamp — a mutable status field kept without history. For such a column the truncated database is indistinguishable from the full one and no execution-based check can decide it. We exclude one such column (`order_status`) from all conditions rather than report an undecidable case as clean.

### C. Why univariate probes are the wrong instrument

If a leaked signal of fixed total strength is spread over  $d$  aggregate features, its visibility to any single-feature statistic shrinks with  $d$  while its effect on a multivariate model does not; with enough aggregates a leak of arbitrary strength stays below any univariate threshold. Fig. 2 shows this empirically; when leakage concentrates in one feature the probe does see it, as on one of our three tasks.

### D. Differential execution and localisation

The check follows from (1): run  $\varphi$  twice and compare.

```
full = program(db, t, entities)
trunc = program(db.truncate(t), t, entities)
verdict = (full != trunc)
```

A divergence is a *proof* of violation: the same program, given strictly less information, produced a different answer — whether through an aggregate, a join, or a statistic fitted on the full table. There are no false positives by construction. The guarantee is one-sided — a violation that does not manifest at the tested seed time is not observed — and the program must be deterministic and re-runnable.

The same primitive localises the leak. The database has a finite set of *availability channels*: “rows of table  $T$  appearing after  $t$ ” and “values of column  $c$  becoming known after  $t$ ”. Truncating one channel at a time and re-running  $\varphi$  names the channels on which the output changes and the output columns each affects; the minimal patch — truncate the input on exactly those channels — is then re-checked. This costs one run per channel (seven in our database) and still reads no code.

## III. SYSTEM

Fig. 1 shows the pipeline. SCOUT holds the availability map — which column governs the availability of which field, and which fields have none. In the current release the map

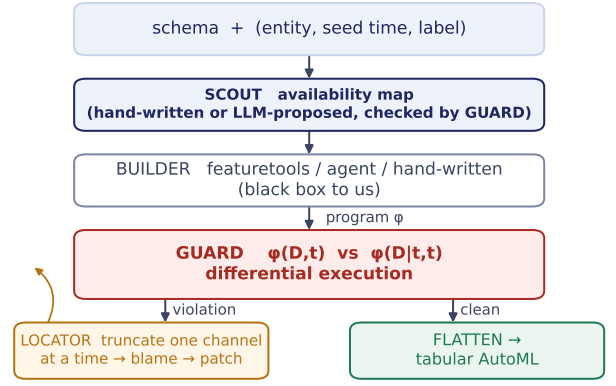


Fig. 1. PITFALL sits between any feature builder and any tabular AutoML. GUARD and LOCATOR are deterministic; the availability map consumed by SCOUT is hand-written in the current release and may be proposed by an LLM.

is a small hand-written declaration (row-time column per table, value-time column per late-written field); an LLM may propose it from the schema — a schema question whose answer is independently checkable, unlike “is there leakage here” [5]. BUILDER is anything that emits a feature program: a library, an agent, a human. GUARD performs the differential check and returns a verdict, the list of diverging columns and the number of diverging cells. On a violation, LOCATOR runs the per-channel truncation, reports which channel feeds which output column, applies the minimal input patch and re-checks. Clean programs are flattened into a single table and handed to an unmodified tabular AutoML system such as AutoGluon [15]. The core is about 170 lines of Python over pandas and numpy, with no dependency on the builder’s language.

**Validating the checker.** On six reference programs with known verdicts (correct; leak in the entity’s own history; leak only through a join; both; no cutoff; and our original “correct” code, see below) at three seed times, the checker was right on 18 of 18 combinations. As a negative control we set the seed time to the maximum timestamp in the database, where every aggregation is correct by construction and the expected result is zero alerts. The first run of that control was not zero: two null outputs were compared as unequal, so programs that returned nothing looked like violations. The control ran before the main experiment, as pre-registered; re-auditing the 23 verdicts issued until then found one false.

## IV. EVIDENCE

**Setup.** Olist [16], a public Brazilian marketplace database: 7 tables, 112,650 order line items, September 2016 to October 2018. Three tasks in RelBench [9], [10] form — seller activity (A), seller review quality (B), product demand (C) — three seed times (January, April, July 2018), 90-day horizon, training on all earlier seed times; 750–870 sellers and 4,500–5,800 products per seed time. One LightGBM configuration with a fixed seed everywhere. Two cores, no GPU.

TABLE I

SOURCES OF POINT-IN-TIME VIOLATION. ROWS 1, 2 AND 5 WERE NOT CONSTRUCTED BY US; ROW 4 IS A CONTROLLED REGIME. “PROBE” IS THE MAXIMUM SINGLE-FEATURE AUC ON THE LEAKING FEATURES; DATA ROBOT WARNS AT 0.85, H2O DRIVERLESS AI AT 0.80.

| Source                        | Inflation, pp   | Probe     | at 0.85 / 0.80      |
|-------------------------------|-----------------|-----------|---------------------|
| featuretools, no cutoff table | +14.8... + 16.3 | 0.76–0.83 | silent / warns 2/3  |
| Our reference code, task B    | +3.1... + 5.3   | 0.62–0.69 | silent / silent     |
| same defect, tasks A and C    | −0.1... + 1.0   | 0.71–0.86 | silent <sup>†</sup> |
| Leak through a join path only | +3.0... + 5.1   | unchanged | silent / silent     |
| LLM code, neutral prompt      | median +0.1     | —         | 53% / 35% violate   |

<sup>†</sup>On task A the probe crosses both thresholds on the *correct* pipeline too (Section IV-C).

### A. The cost of a shifted cutoff

$I(\delta)$  grows monotonically over  $\delta \in [0, 90]$  days on tasks A and B at all three seed times: a one-month error already costs 5.8–11.5 points, and removing the cutoff entirely yields +12.3 to +27.0 (A) and +5.8 to +18.0 (B). For scale: on flat tabular data the mean loss from picking the validation winner among 63 pipelines (16 datasets, 3 seeds) is 0.38 points. On task C we control the cutoff separately for the entity’s own history and for aggregates reached *through a join* to the seller: leaking only through the join costs +3.0 to +5.1 points and, as Section IV-C shows, is invisible to the probe.

### B. Violation is routine

Table I collects the sources.

A library, called as in its introductory example. featuretools 1.31.0 with `ft.dfs` and no cutoff-time table inflates AUC by 14.8, 16.3 and 15.6 points on the three seed times of task C. The library supports cutoff times; it does not signal that they are needed. An independently written harness reproduces the probe values to three decimals (0.834/0.809/0.759).

*Expert code, written by us.* Our reference implementation filtered history by order time and then took all columns of the admitted rows. Two of those columns have their own, later timestamps: the review score and the delivery outcome. Across the three seed times, 5.0%, 5.0% and 2.0% of orders placed before the seed time carry a review written after it. The checker flagged exactly four columns (`review_score_mean`, `review_score_min`, `late_mean`, `delay_days_mean`) and no others. Cost: +3.09, +5.26, +3.78 points on task B, but only +0.16 to +0.31 on A and −0.10 to +0.98 on C.

*Published expert SQL.* In the hand-written feature SQL of the RelBench user study [9] (15 files, 172 joins), a manual audit found two genuine violations, roughly 1% of joins; a syntactic scanner raised eight candidates of which six were false. This audit was static (the database was not reachable from our environment) and we report it as such.

### C. The deployed test does not see it

Fig. 2 plots the probe value against true inflation. The cleanest case is leakage through a join: the probe returns 0.714/0.722/0.717 under a correct cutoff and *the same*

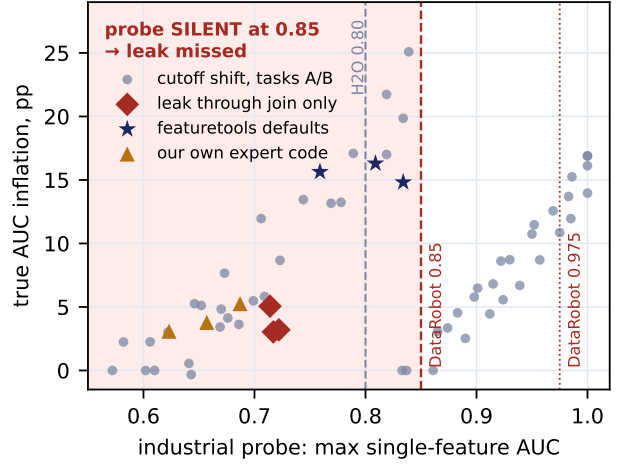


Fig. 2. Probe value against true inflation. Every point left of 0.85 is a violation the DataRobot probe does not report, including all three sources of Table I; the H2O threshold of 0.80 catches two of three featuretools points and none of the others.

0.714/0.722/0.717 under leakage, while the model gains 3.0–5.1 points. On task B no condition — including the total absence of a cutoff, worth +18.0 points — reaches 0.80. The failure runs in both directions: on task A the *correct* pipeline scores 0.837/0.861/0.833, above 0.80 at every seed time and above 0.85 at one, because recency is legitimately the strongest predictor of churn. Lowering the threshold trades misses for false alarms; no threshold separates the two.

### D. Generated feature code

We asked two models — `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code` — for a Python function computing features for task C, giving the schema, the training table with the seed-time column named, and nothing else: the words “leakage” and “point-in-time” do not appear. Each generation is single-shot with a clean context at temperature 0.7; verdicts come from the checker; five of the flagged programs were additionally read by hand and confirmed. Of 40 generations per model, 30 and 17 ran (the rest failed to execute and are excluded, which may bias the rate in either direction). Of those, 53.3% [36.1, 69.8] and 35.3% [17.3, 58.7] were incorrect. The failure is the same in every case examined: the model filters the entity’s own history by the primary event timestamp and does not account for the later timestamp of the joined entity. We make no claim about model capability in general.

Two follow-ups locate the effect. *Task construction* decides it: on a second task with a single seed time per entity, only the entity’s own history and no secondary time axis, both models produce 0 violations in 24 and 28 running programs. *Prompt guards help partially* (30–40 generations per cell, 9–30 running): moving from a neutral prompt to a general reminder to a per-aggregate requirement takes the pooled rate from 46.8% [33, 61] to 41.2% [26, 58] to 14.6% [7, 28] — monotone, but not zero.

SCENE 4 LOCATOR: where exactly it leaks, and a patch

The database is truncated on one availability channel at a time (rows of a table appearing after  $t$ , or a column value becoming known after  $t$ ); a channel on which the program's output changes is a leak path. The program is still not read: as many calls as there are channels.

CLEAN

| AVAILABILITY CHANNEL                     | AFFECTED OUTPUT COLUMNS            | CELLS |
|------------------------------------------|------------------------------------|-------|
| flat.review_score: value known after $t$ | review_score_mean review_score_min | 732   |
| flat.late: value known after $t$         | late_mean                          | 605   |
| flat.delay_days: value known after $t$   | delay_days_mean                    | 677   |

patch: the same program, input truncated on the channels found only - CLEAN

Fig. 3. Scene 4 of the demonstration page: LOCATOR names the availability channel behind each diverging column and re-checks the patched program.

#### E. The cost of a violation is not implied by the violation

The same defect — the same secondary time axis, the same code — costs 0.16–0.31 points on task A, −0.10–0.98 on C and 3.09–5.26 on B; among model-generated programs the median inflation is 0.11 points (range −2.2 to +2.0). Cost depends on how many features are touched and how much the target leans on them, neither visible from the fact of the violation. Correctness therefore has to be tested separately from the quality metric, which an execution-based checker does and a metric-based heuristic cannot.

### V. DEMONSTRATION

The demonstration runs offline on a laptop in about two minutes (`python3 demo.py`), prints per scene a verdict, the diverging columns, the probe value at both thresholds and the true inflation, and renders the same results as a self-contained page (Fig. 3). Attendees can edit a scene’s feature function and re-run the check.

**Scene 1 — library defaults.** `ft.dfs` without a cutoff-time table. Verdict VIOLATION in 7 s, 11 diverging columns, 19,334 cells; probe 0.809 (silent at 0.85, warns at 0.80); inflation +16.3 points.

**Scene 2 — our own code.** Verdict VIOLATION in 0.2 s, naming exactly the four review and delivery aggregates; probe 0.687, silent at both thresholds; +5.3 points. This code was written and read by the authors of the checker.

**Scene 3 — the fix.** The same program with the availability relation fixed and nothing else changed. Verdict CLEAN, outputs identical, inflation 0.00 at all three seed times: the negative control for the first two scenes.

**Scene 4 — localisation.** LOCATOR truncates the seven availability channels one at a time. For Scene 2 it attributes the two review aggregates to `review_score` becoming known after  $t$  and the two delivery aggregates to the delivery date; for Scene 1 it attributes all eleven columns to `items` rows appearing after  $t$ . In both cases the minimal input patch re-checks CLEAN.

### VI. LIMITATIONS

(i) *One-sided.* The check proves violation, never correctness, and only with respect to the observed database and the chosen mask. (ii) *Undecidable columns.* Mutable fields kept without history admit no availability timestamp and cannot be checked by execution at all. (iii) *Dependence on the availability map.*

If the map assigns the wrong timestamp, the wrong thing is checked. (iv) *Scope of measurement.* One database, three tasks, three seed times; two models at one price tier; rates on generated code describe practice under specific prompts at a point in time. (v) *Final-program metric undercounts.* An agent’s validation-driven selection can discard a leaking intermediate trial, as we observed once; checking only the final artifact understates the rate.

### VII. CONCLUSION

Point-in-time correctness in multi-table feature pipelines is violated routinely, and the univariate probe deployed to catch it is silent in exactly the regimes that matter. Differential execution decides the question in seconds without parsing anything, localises the leak by the same primitive, and states the class it cannot decide.

### DATA AND ETHICS

All experiments use the public Olist dataset (Kaggle, CC BY-NC-SA 4.0), anonymised by its publisher; no human subjects were involved. Language-model outputs were used only as objects of measurement, unedited, and are released with the code.

### AVAILABILITY

Code, data and all reported numbers: <https://github.com/stas1f1/pitfall>.

### REFERENCES

- [1] S. Kaufman, S. Rosset, and C. Perlich, “Leakage in data mining: formulation, detection, and avoidance,” in *Proc. KDD*, 2011.
- [2] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023.
- [3] C. Yang *et al.*, “Data leakage in notebooks: static detection and better processes,” in *Proc. ASE*, 2022.
- [4] S. Grafberger, S. Guha, J. Stoyanovich, and S. Schelter, “mlinspect: a data distribution debugger for machine learning pipelines,” in *Proc. SIGMOD*, 2021.
- [5] N. Alturayef and J. Hassine, “Data leakage detection in machine learning code: transfer learning, active learning, or low-shot prompting?” *PeerJ Computer Science*, 11:e2730, 2025.
- [6] C. Isabella, “leakr: data leakage detection,” CRAN package, 2025.
- [7] DataRobot, “Target leakage,” product documentation, <https://docs.datarobot.com>, accessed Aug. 2026.
- [8] H2O.ai, “Driverless AI: leakage detection,” product documentation, <https://docs.h2o.ai>, accessed Aug. 2026.
- [9] J. Robinson *et al.*, “RelBench: a benchmark for deep learning on relational databases,” in *NeurIPS Datasets and Benchmarks*, 2024.
- [10] J. Gu, R. Ranjan, C. Kanatsoulis *et al.*, “RelBench v2: a large-scale benchmark and relational data repository,” in *ICLR Workshop on Foundation Models for Tabular and Structured Data (DATA-FM)*, 2026.
- [11] M. Wang *et al.*, “4DBInfer: a 4D benchmarking toolbox for graph-centric predictive modeling on relational DBs,” in *NeurIPS Datasets and Benchmarks*, 2024.
- [12] “RelAgent: agentic feature engineering for relational data,” arXiv preprint arXiv:2605.07840, 2026. (Preprint; not peer reviewed.)
- [13] J. M. Kanter and K. Veeramachaneni, “Deep feature synthesis: towards automating data science endeavors,” in *Proc. DSAA*, 2015.
- [14] Featuretools, “Handling time,” documentation for v1.31, <https://featuretools.alteryx.com>, accessed Aug. 2026.
- [15] N. Erickson *et al.*, “AutoGluon-Tabular: robust and accurate AutoML for structured data,” arXiv preprint arXiv:2003.06505, 2020.
- [16] Olist, “Brazilian e-commerce public dataset,” Kaggle, 2018.